

Каква е разликата между static и new?

Когато имаме съвсем прост метод, който не изисква нищо допълнително, той е static. Например:

```
internal class Program
{
    0 references
    static void Main(string[] args)
    {
        Crash(); извикваме
    }

    2 references рекурсия
    static void Crash()
    {
        Crash();
    }
}
```

Рекурсията е метод, който вика сам себе си. В случая не прави нищо и само се повтаря докато препълни паметта.

```
D:\Exercises\11 class\01_exercise_27.10.25\bin\Debug\net8.0\01_exercise_27.10.25.exe
Stack overflow.
Repeat 24086 times: в Program.cs
-----
at _01_exercise_27._10._25.Recursions.Crash()
-----
at _01_exercise_27._10._25.Program.Main(System.String[])
```

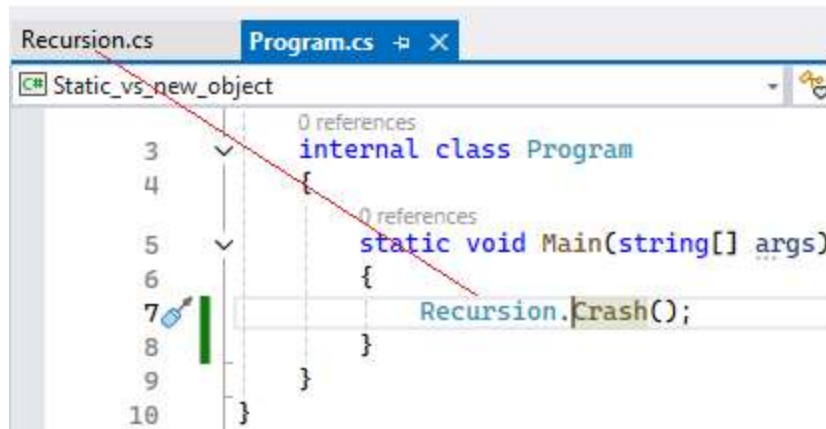
Нека го изрежем с **Ctrl + X** = cut като ножица и поставим **Ctrl + V** = paste в отделен файл в клас, наречен Recursion. Той е прост, статичен. Даваме му модификатор за достъп public, за да бъде видим и достъпен в Main()

```
Recursion.cs  Program.cs
Static_vs_new_object

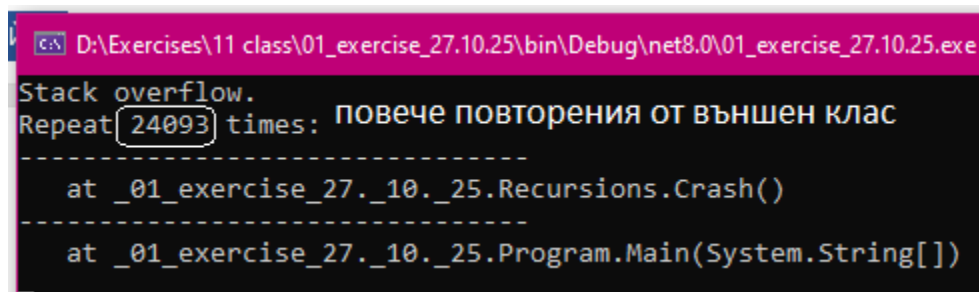
9  0 references
10 internal class Recursion
11 {
12     1 reference paste
13     public static void Crash()
14     {
15         Crash();
16     }
}
```

```
public static void Crash()
{
    Crash();
}
```

Затова, в Main() можем да го извикаме директно по името на класа.метод:



Някои забелязаха, че така повторенията са малко повече на брой.



Причината е, че компилаторът винаги търси първо Program.cs и Main()-метода в него. Ако ние напълним Program.cs с много код, програмата ще се натовари повече и **ще похабим повече процесорни ресурси**, съответно и ще забавим изпълнението, отколкото ако отделим кода на друго място и оставим Main() възможно най-опростен, като съдържание на книга, който само извиква името на метода, който му трябва. Това е един от принципите на оптимизация откъм ресурси – **избягваме блокови {} конструкции в първия стартов клас, който вика всички останали**. Важното е да запомните, че **при най-простия static не се налага да създаваме нов обект с new**.

Казахме, че Обектно-ориентираното програмиране се опитва да пресъздаде реални обекти от заобикалящия свят в програмен вид. За описание на един обект не са достатъчни прости типове данни като int, double, string... Трябват съвкупност от характеристики. Тези характеристики (свойства properties или полета fields) са представени от прости типове данни. Така например шаблонът за обекти клас Person може да се състои от string name, int age, double height и т.н. Когато този шаблон клас е готов, можем да създаваме много обекти по негов тертип. Всеки нов обект е от сложен-съставен тип. Освен характеристиките, обектът има и действия, които в случая са методите (actions).

Нека сега създадем клас за държава, в която защитаваме полетата като private. В предните упражнения видяхме, че не можем да видим private в Main() по никакъв начин. Демонстрацията е,

че можем да видим private, ако използваме междинен public, който извиква в себе си private. Т.е. public се явява посредник. Също, демонстрирам, че винаги има как да се заобиколят правилата чрез Console.ReadLine(), което изобщо не е хубаво, защото няма никакви защиты. Последно, питайте как могат да се подадат данни на private – ами ето така – директно при декларирането правим seeding със стойности. Оставила съм и една уязвимост – подаване на **непроверен (невалидиран)** стринг от конзолата директно: **private string name = Console.ReadLine();** - това въобще не е препоръчително, но е възможно. Винаги може да се създаде уязвимост в една система.

```
public class Country
{
    защитени полета с малка буква
    private string name = Console.ReadLine(); // private field
    private string capital = "София";
    private int population = 6000000;
    private int area = 111000;
    private double frequency;
    невидими директно в Main()

    действие, метод, action
    public void Frequency() // викваме private чрез междинен public
    {
        frequency = (double)population / (double)area;
        Console.WriteLine($"Държава: {name} | Столица: {capital} | Население: {population} | Площ: {area} | Гъстота: {frequency:F4}\n ");
    }
}
```

В нашия клас, имаме защиты с private. Те са невидими в Main() и не могат да бъдат извикани. Създадохме сложен-съставен тип клас Country(), който е съставен от няколко прости типа string, int, double и ги е пакетирал като обект държава. Той има метод, който извършва действие – изчислява гъстотата на населението и отпечатва на конзолата информация. Идеята на задачата е, че макар private да са скрити, ние може да ги видим чрез междинен public (метода Frequency), който ги извиква. По същия начин в предишното изпитване създадохме public-метод, който извика в тялото си друг private-метод за повече защита. Важно е да запомните, че **за да достигнем до private, трябва да имаме междинен public**, който да го обгърне като балон и чрез public да стигнем до private, а не директно.

```
public class Country
{
    private string name = Console.ReadLine(); // private field
    private string capital = "София";
    private int population = 6000000;
    private int area = 111000;
    private double frequency;

    public void Frequency()
    {
        frequency = (double)population / (double)area;
        Console.WriteLine($"Държава: {name} | Столица: {capital} | Население: {population} | Площ: {area} | Гъстота: {frequency:F4}\n ");
    }
}
```

Имайте предвид, че в C# VS, имаме чувствителност и разпознаване на главни и малки букви Key Sensitive. Класовете, методите и public променливите са с главна буква, докато private – с малка. Ще хвърля грешки на Country или country. Има значение. Също, не преименувайте class Program!!! Има само 1 class Program, който държи Main() – компилаторът търси Program и Main() в него. Не бива да ги размествате и пишете Main() в други класове!!!

```

internal class Program
{
    0 references
    static void Main(string[] args)
    {
        //RecurSIONS.Crash(); да не пречи

        Separator();
        Console.WriteLine("=== 1 зад === защита с private");
        Console.Write("Въведете име на държава: (България) ");
        Country country1 = new Country(); не е static, нов обект
        country1.Frequency(); само той е public
    }
}

```

```

//RecurSIONS.Crash();

Separator();
Console.WriteLine("=== 1 зад === защита с private");
Console.Write("Въведете име на държава: (България) ");
Country country1 = new Country();
country1.Frequency();

```

И може би тук е моментът да обясним откъде идва и името на C#. Знаем предците му C и C++ откъде са взели имената си (операторът за инкрементиране i++), но защо и откъде #?

- **C# language name:** The name "C#" is a play on the musical notation "C#" (C sharp), which is a note that is a semitone higher than C. The language was originally named "COOL" (C-like Object Oriented Language) but was changed to "C#" for trademark reasons. 

От музикалната нотация диез, която повишава полутон. Оригиналото наименование на C# е било COOL – C-like Object Oriented Language – C-подобен обектно-ориентиран език. Т.е. в C# насоката са обектите.

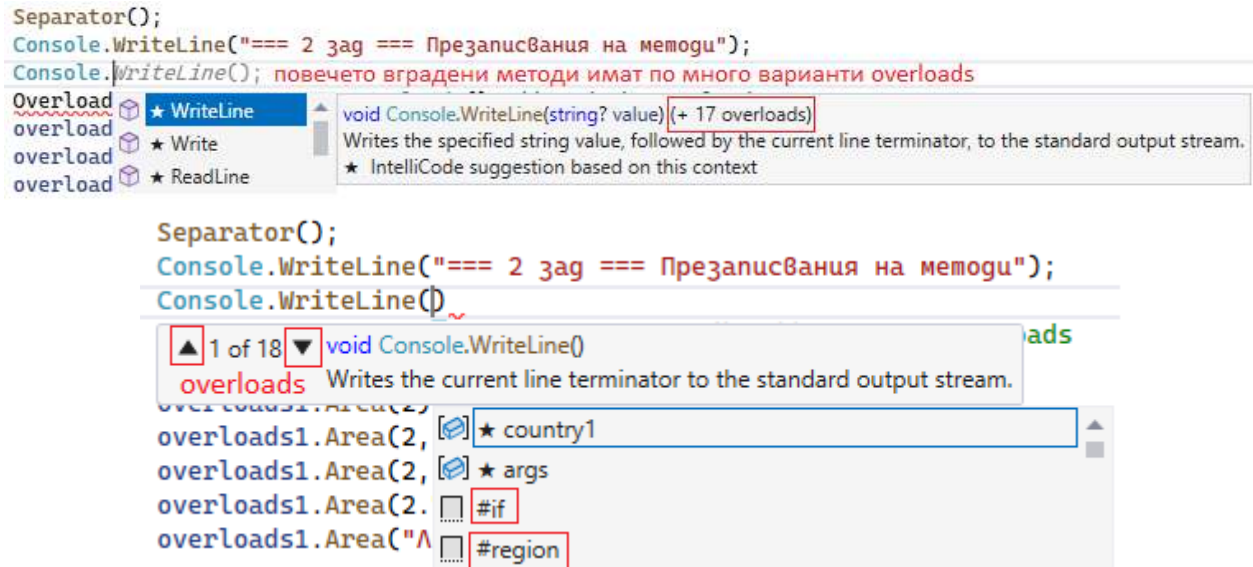
Другото нещо, което се опитам да обясня и предния път е, че # се използва като защитен механизъм за орязване и съкращения – както в ##.## ако имаме 3.40 -> 3.4 изрязване. Този защитен механизъм е заложен и в програмата Excel, при която, ако дадено число е по-дълго от клетката, няма да се среже числото и да се виждат само част от цифрите, защото това ще ни подведе, а се виждат защитни ###. Те сигнализират, че зад тях се крият цифри.

Като цяло, # сигнализира за символ, който се пренебрегва.

Може да направите аналогия със звездичките **** които скриват пароли, но обикновено # скрива цифри (видяхме с форматирането на 0-и).

f_x	123456789	
D	E	F
	###	

Също, ще видим презаписванията на методи – дали може да имаме много методи с едно и също име, например Area, но различен брой променливи и различен тип на тези input променливи между скобите? Дали компилаторът няма да се обърка, ако са с еднакви имена, а с различни променливи? Всъщност, може и ние сме го използвали досега чрез вградените варианти overloads. Например Math.Round() имаше много вградени възможности, при които можеше да подадем само една променлива, която да закръгли като Math.Round(variable), но можеше и да уточним с колко знака след запетаята Math.Round(variable, 2). По подобен начин дори CW има много варианти overloads, а компилаторът не ги греша.



Обърнете внимание, че има и специфичен регион: `#region` и `#endregion`. Те се използват, когато искаме да свием/разгънем определен участък код, за да не ни пречи. Досега свивахме и разгъвахме само пакети методи, класове, функции, конструкции между {}, но може да обхванем по-големи райони, като ги заградим с `#region`. Това е особено удобно в `Main()`, който казахме, че е като съдържание на книга и само извиква имената на методите и им задава стойности. В `Main()` не е препоръчително да има блокови конструкции {}, а само извикване на имена на методи, съответно там скриване и разгъване на код по-трудно може да се осъществи.

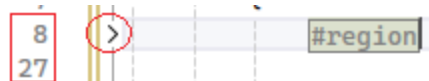


```

6      static void Main(string[] args)
7      {
8          #region
9          //Recursions.Crash();
10
11         Separator();
12         Console.WriteLine("=== 1 заг === защита с private");
13         Console.Write("Въвегеме име на държава: (България) ");
14         Country country1 = new Country();
15         country1.Frequency();
16
17         Separator();
18         Console.WriteLine("=== 2 заг === Презапусвания на мемогу");
19         Overloads overloads1 = new Overloads(); // Methods Overloads
20         overloads1.Area();
21         overloads1.Area(2);
22         overloads1.Area(2, 3);
23         overloads1.Area(2, 3, 4);
24         overloads1.Area(2.7);
25         overloads1.Area("Лице и площ са синоними.");
26         #endregion

```

И съответно свием:



Това е една от екстрите на C#.

Другата е **#if**. Когато имаме много дълъг код, който искаме да проверим с дебъгера, но не искаме да се изпълнява целия последователно, защото вече огромна част от него е вярна и само ще ни забави, използваме **#if** - препроцесорна директива, която се изпълнява преди компилацията. Тя определя кой код ще бъде компилиран.

#if УСЛОВИЕ

// Код, който се компилира само ако е вярно

#else

// Код, който се компилира ако не е вярно

#endif

Това е една от функционалностите, които се използват при по-големи проекти с повече настройки като **#define** с ключова дума в началото на файла, затова няма да я използваме засега, но като ориентир имаме дефиниране и после разделяне на участъци с нея.


```
Reverse.cs  Program.cs  Overloads.cs  GCD_LCM.cs  Sort.cs
rcise_27.10.25  _01_exercise_27.10.25.Program
1  #define COUNTRY // Дефиниране в началото на файла
2  #define AREA
3
4  namespace _01_exercise_27._10._25
5  {
6      internal class Program
7      {
8          static void Main(string[] args)
9          {
10             #region
11             //Recursions.Crash();
12             #if COUNTRY
13             Separator();
14             Console.WriteLine("=== 1 заг === защита с private");
15             Console.WriteLine("Въвегеме име на държава: (България) ");
16             Country country1 = new Country();
17             country1.Frequency();
18             #endif
19             #if AREA
20             Separator();
21             Console.WriteLine("=== 2 заг === Презареждането на метогу");
22             Overloads overloads1 = new Overloads(); // Methods Overloads
23             overloads1.Area();
24             overloads1.Area(2);
25             overloads1.Area(2, 3);
26             overloads1.Area(2, 3, 4);
27             overloads1.Area(2.7);
28             overloads1.Area("Лице и площ са синоними.");
29             #endif
30             #endregion
          }
      }
  }
```

И да се върнем на новия материал за overloads – презареждането с имена. Ако погледнете кода, в клас Overloads.cs съм създавала много методи, които се казват по един и същи начин Area(), но между скобите се подават различен брой променливи, при това от различен тип. Така компилаторът се ориентира кой точно метод е извикан и го замества във формулата (дефиницията) на метода.

```
Country.cs | Overloads.cs | Program.cs | GCD_LCM.cs
e_27.10.25 | 01_exercise_27.

2 references
internal class Overloads
{
    1 reference
    public void Area()
    {
        Console.WriteLine("Демонстрация на презареждане с имена: ");
    }
    1 reference
    public void Area(int a)
    {
        Console.WriteLine($"Лице на квадрат: {a*a}");
    }
    1 reference
    public void Area(int a, int b)
    {
        Console.WriteLine($"Лице на правоъгълник: {a*b}");
    }
    1 reference
    public void Area(int a, int b, int c)
    {
        double p = (double)(a + b + c)/2;
        double s = Math.Sqrt(p*(p-a)*(p-b)*(p-c)); // square root
        Console.WriteLine($"Лице на триъгълник: {s}");
    }
    1 reference
    public void Area(double r)
    {
        Console.WriteLine($"Лице на кръг: {2*Math.PI*r:F2}");
    }
    1 reference
    public void Area(string message)
    {
        Console.WriteLine("Добре изпълнени задачи с лица!");
        Console.WriteLine("Вашият поздрав е: " + message);
    }
}
```

В случая не са static, значи ще изискват създаване на нов обект в Main():

```
internal class Overloads
{
    public void Area()
    {
        Console.WriteLine("Демонстрация на презареждане с имена: ");
    }
    public void Area(int a)
    {
        Console.WriteLine($"Лице на квадрат: {a*a}");
    }

    public void Area(int a, int b)
    {
        Console.WriteLine($"Лице на правоъгълник: {a*b}");
    }
    public void Area(int a, int b, int c)
    {
        double p = (double)(a + b + c)/2;
```



```

        double s = Math.Sqrt(p*(p-a)*(p-b)*(p-c)); // square root
        Console.WriteLine($"Лице на триъгълник: {s}");
    }
    public void Area(double r)
    {
        Console.WriteLine($"Лице на кръг: {2*Math.PI*r:F2}");
    }
    public void Area(string message)
    {
        Console.WriteLine("Добре изпълнени задачи с лица!");
        Console.WriteLine("Вашият поздрав е: " + message);
    }
}

```

Съответно създаваме нов обект в Main() и подаваме различни стойности на променливите за публичните методи, за да тестваме имплементацията им.

```

#region
//RecurSIONs.Crash();
#if COUNTRY
    Separator();
    Console.WriteLine("=== 1 зад === защита с private");
    Console.WriteLine("Въведете име на държава: (България) ");
    Country country1 = new Country();
    country1.Frequency();
#endif
#if AREA
    Separator();
    Console.WriteLine("=== 2 зад === Презареждания на методи");
    Overloads overloads1 = new Overloads(); // Methods Overloads
    overloads1.Area();
    overloads1.Area(2);
    overloads1.Area(2, 3);
    overloads1.Area(2, 3, 4);
    overloads1.Area(2.7);
    overloads1.Area("Лице и площ са синоними.");
#endif
#endregion

```

```
overloads1.Area(2.7);
```

```
overloads1.Area("Лице и площ са синоними.");
```

```
overloads1.    Нашият Area, който ние създадохме, а не е вграден има 5 overloads
```

```

#endregion
    Area
    Equals
    GetHashCode

```

Можем със стрелкичките да разглеждаме, четем и изберем кой точно overload искаме да използваме.

```

28      overloads1.Area("Лице и площ са синоними.");
29      overloads1.Area()
30  #endif
31  #endif
32
33      Separator();
34      Console.WriteLine("Рекурсия");
35      Recursions recursion1 = new Recursion1();
36      Console.WriteLine("Факториал на 6 = {0}", recursion1.Factorial(6));
37      Console.WriteLine("Степен на 2 ^ 3 = {0}", recursion1.Power(2, 3));
38
28      overloads1.Area("Лице и площ са синоними.");
29      overloads1.Area()
30  #endif
31  #endif
32
33      Separator();
34      Console.WriteLine("Рекурсия");
35      Recursions recursion1 = new Recursion1();
36      Console.WriteLine("Факториал на 6 = {0}", recursion1.Factorial(6));
37      Console.WriteLine("Степен на 2 ^ 3 = {0}", recursion1.Power(2, 3));
38
28      overloads1.Area("Лице и площ са синоними.");
29      overloads1.Area()
30  #endif
31  #endif
32
33      Separator();
34      Console.WriteLine("Рекурсия");
35      Recursions recursion1 = new Recursion1();
36      Console.WriteLine("Факториал на 6 = {0}", recursion1.Factorial(6));
37      Console.WriteLine("Степен на 2 ^ 3 = {0}", recursion1.Power(2, 3));
38

```

▲ 1 of 6 ▼ void Overloads.Area()

OverflowException
Overlapped
Overloads
overloads1
Parallel
ParallelEnumerable

▲ 2 of 6 ▼ void Overloads.Area(double r)

OverflowException
Overlapped
Overloads
overloads1

▲ 4 of 6 ▼ void Overloads.Area(string message)

OverflowException
Overlapped
Overloads
overloads1
Parallel

Последно – рекурсия (метод, който извиква себе си) и сравнение на static и new

```
try.cs Recursions.cs Overloads.cs
2 references
public class Recursions
{
    не са static -> трябва им нов обект
    2 references
    public int Factorial(int f)
    {
        if (f <= 1) return 1;
        return f * Factorial(f - 1);
    }
    2 references double
    public int Power(int a, int b)
    {
        if (b == 0) return 1;
        if (b == 1) return a;
        return a * Power(a, b - 1);
    }
    1 reference може директно без new
    public static void Crash()
    {
        Recursions.Crash(); в Main()
        Crash();
    }
}
```

В примера с рекурсиите съм показала само факториел и повдигане на положителна степен (без дроби). Тук идеята е отново да се съпоставят методи със static (могат да се извикат директно в Main()) по име на файл.метод и без static – изискват нов обект.

```
public class Recursions
{
    public int Factorial(int f)
    {
        if (f <= 1) return 1;
        return f * Factorial(f - 1);
    }
    public double Power(int a, int b)
    {
        if (b == 0) return 1;
        if (b == 1) return a;
        return a * Power(a, b - 1);
    }
    public static void Crash()
    {
        Crash();
    }
}
```

```
Recursions.cs    Country.cs    Overloads.cs    Program.cs  GCD_LCM.cs    So
10.25 static директно  _01_exercise_27_10_25.Program
Separator();
Console.WriteLine("=== 3 зад === Рекурсии");
Recursions recursion1 = new Recursions(); // new object
Console.WriteLine($"
Factorial of 6 = {recursion1.Factorial(6)}
Power of 2 ^ 3 = {recursion1.Power(2, 3)}");
//Recursions.Crash(); ако махнем static, ще бъде част от обекта
//recursion1.Crash(); без static изисква new -> recursion1.
```

```
Separator();
Console.WriteLine("=== 3 зад === Рекурсии");
Recursions recursion1 = new Recursions(); // new object
Console.WriteLine($"
Factorial of 6 = {recursion1.Factorial(6)}
Power of 2 ^ 3 = {recursion1.Power(2, 3)}");
//Recursions.Crash(); ако махнем static, ще бъде част от обекта
//recursion1.Crash();
```

Като цяло рекурсиите са рискови откъм безкрайни извиквания, а в двата примера дори излишни, т.к. за изчисляване на степен имаме готов вграден метод `Math.Pow(2,-3)`, който работи и с отрицателни и дробни числа, а факториел може да се изчисли дори с цикъл `for()`.

За упражнение: Създайте 2 метода в клас `Overloads`, които имат еднакви имена `Perimeter`, но единият изчислява обиколка на правоъгълник, а другият на триъгълник. Нека променливите и резултатът бъдат от тип `double`. Извикайте ги в `Main()` и ги отпечатайте на конзолата.

За оценка: Създайте 2 метода в клас `Overloads`, които имат еднакви имена `Volume`, но единият изчислява обем на куб, а другият на паралелепипед. Нека променливите и резултатът бъдат от тип `double` закръглени до 3-я знак след десетичната запетая. Извикайте ги в `Main()` и ги отпечатайте на конзолата.